Lec-23: Translation Units, Linkage & ODR

Why Code Lives in Many Files

- **Incremental builds** — only recompile what changed, not the whole project.

Why Code Lives in Many Files

- **Incremental builds** — only recompile what changed, not the whole project.
- **Parallel work** — two developers can edit separate files without stepping on each other.

Why Code Lives in Many Files

- **Incremental builds** — only recompile what changed, not the whole project.
- **Parallel work** — two developers can edit separate files without stepping on each other.
- **Libraries** — reusable code is shipped as a compiled package; callers link against it without having the source.

Why Code Lives in Many Files

- **Incremental builds** — only recompile what changed, not the whole project.
- **Parallel work** — two developers can edit separate files without stepping on each other.
- **Libraries** — reusable code is shipped as a compiled package; callers link against it without having the source.

Key question: When code lives in separate files, *how does one file "see" something defined in another* — and what are the rules?

#include is a Copy-Paste

Before the compiler sees a single line, the **preprocessor** runs:

```
#include "student.h"   // replaced with the full text of student.h
#include <iostream>    // replaced with the full text of iostream
int main() { /* ... */ }
```

#include is a Copy-Paste

Before the compiler sees a single line, the **preprocessor** runs:

```
#include "student.h"   // replaced with the full text of student.h
#include <iostream>    // replaced with the full text of iostream
int main() { /* ... */ }
```

#include is literally a textual paste.
The compiler then sees **one long text file**, not two.

#include is a Copy-Paste

Before the compiler sees a single line, the **preprocessor** runs:

```
#include "student.h"   // replaced with the full text of student.h
#include <iostream>    // replaced with the full text of iostream
int main() { /* ... */ }
```

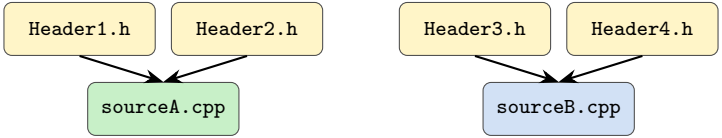
#include is literally a textual paste.
The compiler then sees **one long text file**, not two.

Key: **Declaring** something is *not* the same as **defining** it.

Output preprocessing

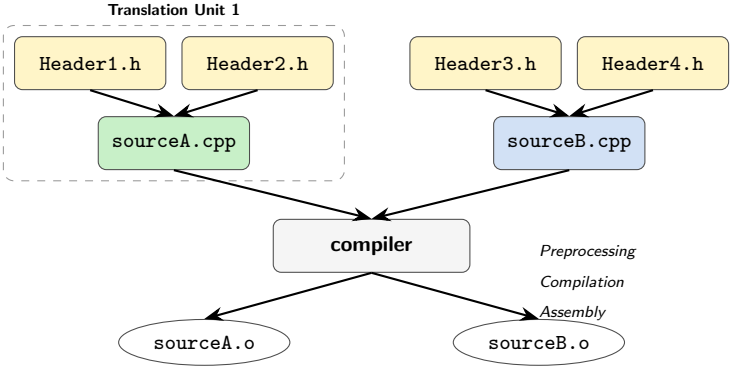
```
g++ -E source.cpp
```

The Compile Pipeline — Step 1: Sources



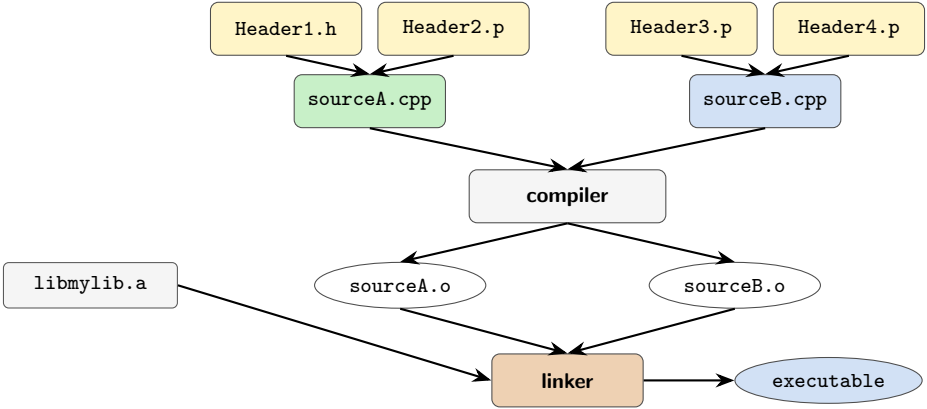
Two `.cpp` files, each pulling in its own headers.

The Compile Pipeline — Step 2: Compile



Each `.cpp` + its headers becomes one `.o` file.

The Compile Pipeline — Step 3: Link



The **linker** stitches `.o` files (and libraries) into the final program.

Translation Unit

A **translation unit** (TU) is a single `.cpp` file together with every header it directly or indirectly includes. It is the smallest unit of compilation: **one** `.cpp` $\rightarrow$ **one** `.o`.

Translation Unit

A **translation unit** (TU) is a single `.cpp` file together with every header it directly or indirectly includes. It is the smallest unit of compilation: **one** `.cpp` → **one** `.o`.

Two important facts:

1. The compiler processes each TU in **complete isolation** — it knows nothing about the other `.cpp` files while compiling yours.

Translation Unit

A **translation unit** (TU) is a single `.cpp` file together with every header it directly or indirectly includes. It is the smallest unit of compilation: **one** `.cpp` → **one** `.o`.

Two important facts:

1. The compiler processes each TU in **complete isolation** — it knows nothing about the other `.cpp` files while compiling yours.
2. The **linker** resolves cross-file references *after* all `.o` files exist.

Quick Check — How Many Translation Units?

main.cpp

```cpp
#include "student.h"
#include "course.h"
int main() { /* ... */ }
```

student.cpp

```cpp
#include "student.h"
void Student::enroll() { /* ... */ }
```

How many translation units?

Quick Check — How Many Translation Units?

main.cpp

```cpp
#include "student.h"
#include "course.h"
int main() { /* ... */ }
```

student.cpp

```cpp
#include "student.h"
void Student::enroll() { /* ... */ }
```

How many translation units?

Two. One from `main.cpp` (sees `student.h` + `course.h`), one from `student.cpp` (sees `student.h`).

Quick Check — How Many Translation Units?

main.cpp

```cpp
#include "student.h"
#include "course.h"
int main() { /* ... */ }
```

student.cpp

```cpp
#include "student.h"
void Student::enroll() { /* ... */ }
```

How many translation units?

Two. One from `main.cpp` (sees `student.h` + `course.h`), one from `student.cpp` (sees `student.h`).

Notice: `student.h` is pasted into *both* TUs — the header is compiled **twice**. That has consequences.

Scope — A Quick Refresher

Scope is the region of source code where a name is *visible*.

```
int global_score = 0;    // global scope: visible everywhere in this file

void addPoints(int n) {
    int bonus = 10;      // local scope: only inside addPoints()
    global_score += n + bonus;
}
// 'bonus' is NOT accessible here
```

Scope — A Quick Refresher

Scope is the region of source code where a name is *visible*.

```
int global_score = 0;    // global scope: visible everywhere in this file

void addPoints(int n) {
    int bonus = 10;      // local scope: only inside addPoints()
    global_score += n + bonus;
}
// 'bonus' is NOT accessible here
```

Scope	Where declared	Lifetime
Local / block	inside { }	until closing }
Global / file	outside all functions	whole program

What Is a Symbol?

When the compiler builds `.o`, every named entity becomes a **symbol** — an entry in the object file's symbol table.

Symbol name	**Kind**	**Address in `.o`**
`main`	function	0x0000_1000
`global_score`	variable	0x0000_2010
`Student::enroll`	function	0x0000_2500

What Is a Symbol?

When the compiler builds `.o`, every named entity becomes a **symbol** — an entry in the object file's symbol table.

Symbol name	**Kind**	**Address in `.o`**
`main`	function	0x0000_1000
`global_score`	variable	0x0000_2010
`Student::enroll`	function	0x0000_2500

The linker reads these tables to connect callers in one `.o` to definitions in another.

Linkage — Three Levels

- **No linkage** — symbol lives entirely inside its block.
  Linker *never* sees it. *(e.g. a local variable.)*

Linkage — Three Levels

- **No linkage** — symbol lives entirely inside its block.
  Linker *never* sees it. *(e.g. a local variable.)*
- **Internal linkage** — visible only within its own TU.
  Each TU gets its own *private* copy; same name in two files does **not** conflict. *(e.g. a* `static` *global.)*

Linkage — Three Levels

- **No linkage** — symbol lives entirely inside its block.
  Linker *never* sees it. *(e.g. a local variable.)*
- **Internal linkage** — visible only within its own TU.
  Each TU gets its own *private* copy; same name in two files does **not** conflict. *(e.g. a* `static` *global.)*
- **External linkage** — visible to *all* TUs.
  Exactly **one** shared copy across the program.
  Same name defined in two files **will** conflict. *(e.g. a regular global or function.)*

Linkage Example

```
void foo() {
    int x = 5;            // NO linkage --- local, linker never sees it
}

int global_score = 10;    // EXTERNAL --- visible to all .cpp files

static int hidden = 20;   // INTERNAL --- private to this file only

void bar() {}             // EXTERNAL --- any file can call bar()

static void helper() {}   // INTERNAL --- private to this file only
```

Linkage Example

```
void foo() {
    int x = 5;            // NO linkage --- local, linker never sees it
}

int global_score = 10;    // EXTERNAL --- visible to all .cpp files

static int hidden = 20;   // INTERNAL --- private to this file only

void bar() {}             // EXTERNAL --- any file can call bar()

static void helper() {}   // INTERNAL --- private to this file only
```

Rule of thumb

If it is declared outside all functions and not marked `static`, the linker can see it from any `.cpp` in the program.

Storage Duration

Every variable has a **storage duration** governing its lifetime, *independent of scope*:

Duration	**Allocated at**	**Deallocated at**	**Example**
Automatic	block entry	block exit	local `int x`
Static	program start	program end	global or `static` var
Dynamic	`new`	`delete`	heap allocation
Thread	thread start	thread end	`thread_local` (C++11)

Storage Duration

Every variable has a **storage duration** governing its lifetime, *independent of scope*:

Duration	**Allocated at**	**Deallocated at**	**Example**
Automatic	block entry	block exit	local `int x`
Static	program start	program end	global or `static` var
Dynamic	`new`	`delete`	heap allocation
Thread	thread start	thread end	`thread_local` (C++11)

- **Automatic** — uninitialised ⇒ undefined value.
- **Static** — zero-initialised before `main` runs.

Storage Duration — Examples

```
int v1;                     // static duration (global)

void f() {
    int  v2;                // automatic duration
    auto v3    = 3;         // automatic duration
    auto array = new int[10]; // dynamic duration (allocation)
}   // v2, v3, 'array' variable freed; but int[10] still on heap!

int main() {
    f();
}   // main end: v1 is deallocated
```

Storage Duration — Examples

```
int v1;                     // static duration (global)

void f() {
    int  v2;                // automatic duration
    auto v3    = 3;         // automatic duration
    auto array = new int[10]; // dynamic duration (allocation)
}   // v2, v3, 'array' variable freed; but int[10] still on heap!

int main() {
    f();
}   // main end: v1 is deallocated
```

Warning: deallocating `array` (the pointer) does *not* free the `new int[10]` — that leaks unless you `delete[]`.

`static` — Two Very Different Meanings

Watch out!

The keyword `static` means **different things** depending on *where* you write it.

- On a **global** variable or function:
  restricts *visibility* $\rightarrow$ **internal linkage**.

`static` — Two Very Different Meanings

Watch out!

The keyword `static` means **different things** depending on *where* you write it.

- On a **global** variable or function:
  restricts *visibility* $\rightarrow$ **internal linkage**.
- On a **local** variable:
  changes *lifetime* $\rightarrow$ **static duration**
  (persists between calls; visibility is still local).

static on a Global — Internal Linkage

```
static int counter = 0;   // only this .cpp file can access counter
static void reset() {}    // only this .cpp file can call reset()
```

static on a Global — Internal Linkage

```
static int counter = 0;    // only this .cpp file can access counter
static void reset() {}     // only this .cpp file can call reset()
```

Two developers each write a helper called `clamp`:

math.cpp

```
// Clamps to [0, 1]
static float clamp(float v) {
    if (v < 0.0f) return 0.0f;
    if (v > 1.0f) return 1.0f;
    return v;
}
```

render.cpp

```
// Clamps to [-1, 1]
static float clamp(float v) {
    if (v < -1.0f) return -1.0f;
    if (v >  1.0f) return  1.0f;
    return v;
}
```

`static` on a Global — Internal Linkage

```
static int counter = 0;    // only this .cpp file can access counter
static void reset() {}     // only this .cpp file can call reset()
```

Two developers each write a helper called `clamp`:

math.cpp

```
// Clamps to [0, 1]
static float clamp(float v) {
    if (v < 0.0f) return 0.0f;
    if (v > 1.0f) return 1.0f;
    return v;
}
```

render.cpp

```
// Clamps to [-1, 1]
static float clamp(float v) {
    if (v < -1.0f) return -1.0f;
    if (v >  1.0f) return  1.0f;
    return v;
}
```

`static` gives each file its own *private* `clamp`. Without it ⇒ **multiple definition** linker error.

static on a Local — Persistent Lifetime

```
int nextId() {
    static int id = 0;   // initialized ONCE; survives between calls
    return ++id;
}

int main() {
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
}
```

static on a Local — Persistent Lifetime

```
int nextId() {
    static int id = 0;   // initialized ONCE; survives between calls
    return ++id;
}

int main() {
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
}
```

Output: 1, 2, 3

`static` on a Local — Persistent Lifetime

```
int nextId() {
    static int id = 0;   // initialized ONCE; survives between calls
    return ++id;
}

int main() {
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
    std::cout << nextId() << "\n";  // ?
}
```

Output: 1, 2, 3

`id` is initialised *once* — on the first call — and its value **persists** to program end.
The *name* is still visible only inside `nextId`.

extern

`extern` *declares* that a symbol is **defined in another translation unit**. It does **not** allocate memory.

main.cpp

```cpp
extern int score;           // "score" exists elsewhere
extern void printScore();   // extern is optional for functions

int main() {
    score = 42;
    printScore();
}
```

game.cpp

```cpp
#include <iostream>
int score = 0;              // ACTUAL memory here

void printScore() {
    std::cout << score << "\n";
}
```

extern

`extern` *declares* that a symbol is **defined in another translation unit**. It does **not** allocate memory.

main.cpp

```
extern int score;           // "score" exists
elsewhere
extern void printScore();   // extern is
optional for functions

int main() {
    score = 42;
    printScore();
}
```

game.cpp

```
#include <iostream>
int score = 0;              // ACTUAL memory
here

void printScore() {
    std::cout << score << "\n";
}
```

Without `extern`, `int score;` in `main.cpp` would *define a second* global `score` ⇒ linker error.

Declaration vs. Definition

- **Declaration** — "this thing *exists* somewhere; here is its name and type."
  No memory allocated. May appear any number of times.

Declaration vs. Definition

- **Declaration** — "this thing *exists* somewhere; here is its name and type."
  No memory allocated. May appear any number of times.
- **Definition** — "here is the *thing itself*."
  Memory allocated (variables) or code emitted (functions). Must appear exactly once — with exceptions we'll cover.

Declaration vs. Definition

- **Declaration** — "this thing *exists* somewhere; here is its name and type."
  No memory allocated. May appear any number of times.
- **Definition** — "here is the *thing itself*."
  Memory allocated (variables) or code emitted (functions). Must appear exactly once — with exceptions we'll cover.

Code	**Declaration?**	**Definition?**
`extern int x;`	✓	×
`int x = 5;`	✓	✓
`void f();`	✓	×
`void f() { ... }`	✓	✓

const at Global Scope ⇒ Internal Linkage

```
// constants.h
const     int   MAX_STUDENTS  = 100;     // INTERNAL linkage
constexpr float PASSING_GRADE = 50.0f;   // INTERNAL linkage
```

const at Global Scope ⇒ Internal Linkage

```
// constants.h
const     int   MAX_STUDENTS  = 100;     // INTERNAL linkage
constexpr float PASSING_GRADE = 50.0f;   // INTERNAL linkage
```

If five `.cpp` files include `constants.h`, each gets its own **private copy**.
The *address* of `MAX_STUDENTS` differs between files (*code bloat*).

const at Global Scope ⇒ Internal Linkage

```
// constants.h
const     int   MAX_STUDENTS  = 100;     // INTERNAL linkage
constexpr float PASSING_GRADE = 50.0f;   // INTERNAL linkage
```

If five `.cpp` files include `constants.h`, each gets its own **private copy**.
The *address* of `MAX_STUDENTS` differs between files (*code bloat*).

Why this default? So you can safely put `const` values in headers *without* linker collisions.

`const` at Global Scope ⇒ Internal Linkage

```
// constants.h
const     int   MAX_STUDENTS  = 100;     // INTERNAL linkage
constexpr float PASSING_GRADE = 50.0f;   // INTERNAL linkage
```

If five `.cpp` files include `constants.h`, each gets its own **private copy**.
The *address* of `MAX_STUDENTS` differs between files (*code bloat*).

Why this default? So you can safely put `const` values in headers *without* linker collisions.

C++17 fix — one shared copy across the whole program:

```
inline constexpr float PASSING_GRADE = 50.0f; // one address, all TUs
```

One Definition Rule (ODR) — Within a Translation Unit

Rule 1

A template, type, function, or object **cannot have more than one definition** within a single translation unit.

- Multiple **declarations** are perfectly fine
- Multiple **definitions** ⇒ compiler error

Example

```
int x;        // definition
int x;        // error: redefinition of 'x'

void f();     // declaration
void f();     // OK: repeated declaration
void f() {}   // definition
void f() {}   // error: redefinition of 'f'
```

One Definition Rule (ODR) — Across the Program

Rule 2

An object or **non-inline function** cannot have more than one definition across all translation units.

- Multiple definitions ⇒ **linker error**
- Entities with **internal linkage** are exempt
  - Each translation unit gets its own independent copy
  - Even if the names and types are identical

Internal Linkage — No Conflict

```
// a.cpp
static int count = 0;

// b.cpp
static int count = 0;

// OK: each TU has its own 'count'
```

One Definition Rule (ODR) — Allowed Multiple Definitions

Rule 3

Some entities **may** be defined in more than one translation unit, provided every definition is **identical**.

Entities that qualify:

- Class / struct / enum types
- Templates (class and function)
- `inline` functions and variables

Warning

If the definitions differ ⇒ **undefined behavior** (not always caught by the compiler!)

Why This Matters

Headers are included in multiple translation units.
Without this exception, placing a class definition in a header would violate ODR.

The Problem ODR Solves

The linker's job: match every *use* of a symbol to *exactly one definition*.
What if it finds **two definitions** of the same name?

The Problem ODR Solves

The linker's job: match every *use* of a symbol to *exactly one definition*.
What if it finds **two definitions** of the same name?

Scenario A: Both `main.cpp` and `game.cpp` define `void update()`. Linker cannot choose.

The Problem ODR Solves

The linker's job: match every *use* of a symbol to *exactly one definition*.
What if it finds **two definitions** of the same name?

Scenario A: Both `main.cpp` and `game.cpp` define `void update()`. Linker cannot choose.

Scenario B: A function *body* is placed in a header. Two `.cpp` files include it. Both TUs now contain a definition — linker sees the name twice.

The Problem ODR Solves

The linker's job: match every *use* of a symbol to *exactly one definition*.
What if it finds **two definitions** of the same name?

Scenario A: Both `main.cpp` and `game.cpp` define `void update()`. Linker cannot choose.

Scenario B: A function *body* is placed in a header. Two `.cpp` files include it. Both TUs now contain a definition — linker sees the name twice.

The ODR is the standard's precise answer.
It tells you exactly how many definitions are allowed, and where.

ODR — Point 1

Point 1 (within one translation unit)

Any type, function, variable, or template may be defined **at most once** within a single TU.

Any number of *declarations* are allowed.

ODR — Point 1

Point 1 (within one translation unit)

Any type, function, variable, or template may be defined **at most once** within a single TU.
Any number of *declarations* are allowed.

Who catches it? The **compiler** — immediately.

ODR — Point 1

Point 1 (within one translation unit)

Any type, function, variable, or template may be defined **at most once** within a single TU.
Any number of *declarations* are allowed.

Who catches it? The **compiler** — immediately.

```
rror:
void display()  void display()  // (*@error:@*) redefinition of 'display'
```

ODR — Point 2

Point 2 (across the whole program)

Non-`inline` functions and non-`inline` variables may be defined in **at most one** translation unit in the whole program.

ODR — Point 2

Point 2 (across the whole program)

Non-`inline` functions and non-`inline` variables may be defined in **at most one** translation unit in the whole program.

Who catches it? The **linker** — "multiple definitions".

ODR Point 2 — Linker Catches This

main.cpp

```cpp
#include <iostream>

int score = 0;      // definition

void display();     // declaration only

int main() { display(); }
```

game.cpp

```cpp
int score = 99;    // LINKER ERROR:
                   // multiple definitions
                   // of 'score'

void display() {
    // which score?
}
```

ODR Point 2 — Linker Catches This

main.cpp

```cpp
#include <iostream>

int score = 0;      // definition

void display();     // declaration only

int main() { display(); }
```

game.cpp

```cpp
int score = 99;    // LINKER ERROR:
                   // multiple definitions
                   // of 'score'

void display() {
    // which score?
}
```

Both files define `score` with external linkage.

Fix: define `score` in exactly one `.cpp`; use `extern int score;` in all others.

ODR — Point 3

Point 3 (the exception for types and `inline`)

Types, `inline` functions, and `inline` variables *may* be defined in **more than one** translation unit — provided **every definition is identical**.

ODR — Point 3

Point 3 (the exception for types and `inline`)

Types, `inline` functions, and `inline` variables *may* be defined in **more than one** translation unit — provided **every definition is identical**.

Who catches it? Nobody.
Violations are *undefined behaviour*

ODR Exception and Undefined Behaviour

Example (ODR violation):

```
// header.h
inline int f() { return 1; } // version A
```

```
// file1.cpp
#include "header.h"
```

```
// file2.cpp
inline int f() { return 2; } // version B (different!)
```

Problem:

- Same inline function, different definitions
- May compile and link
- Behaviour is undefined

The Classic Header Mistake

utils.h

```cpp
#include <iostream>
void greet() {                    // DEFINITION in a header!
    std::cout << "Hello\n";
}
```

main.cpp

```cpp
#include "utils.h"
int main() { greet(); }
```

gui.cpp

```cpp
#include "utils.h"
void showMenu() { greet(); }
```

The Classic Header Mistake

utils.h

```cpp
#include <iostream>
void greet() {                    // DEFINITION in a header!
    std::cout << "Hello\n";
}
```

main.cpp

```cpp
#include "utils.h"
int main() { greet(); }
```

gui.cpp

```cpp
#include "utils.h"
void showMenu() { greet(); }
```

The header is pasted into *both* `.cpp` files, so both TUs get a definition of `greet` $\Rightarrow$ linker fails.

`inline` — "Safe to Define in Headers"

`inline` keyword

"This symbol may appear in multiple TUs. All copies are identical. Linker, keep exactly one."

utils.h

```
// Now safe to include in any number of .cpp files:
inline void greet() { std::cout << "Hello\n"; }

inline int MAX_SIZE = 100;        // C++17: inline variable
```

`inline` — "Safe to Define in Headers"

`inline` keyword

"This symbol may appear in multiple TUs. All copies are identical. Linker, keep exactly one."

utils.h

```
// Now safe to include in any number of .cpp files:
inline void greet() { std::cout << "Hello\n"; }

inline int MAX_SIZE = 100;        // C++17: inline variable
```

Common misconception: `inline` does *not* force the compiler to inline the call, nor does it make code faster.
The word is overloaded; here it means **"safe in headers."**

inline Example

```
        void f() {}
inline void g() {}
```

`inline` Example

```
        void f() {}
inline void g() {}
```

`f()` — plain function:

- *Cannot* be defined in a header included in multiple sources.
- Linker issues "multiple definitions".

`inline` Example

```
        void f() {}
inline void g() {}
```

`f()` — plain function:

- *Cannot* be defined in a header included in multiple sources.
- Linker issues "multiple definitions".

`g()` — `inline`:

- *Can* be defined in a header and included in many sources.

`inline` Example

```
        void f() {}
inline void g() {}
```

`f()` — plain function:

- *Cannot* be defined in a header included in multiple sources.
- Linker issues "multiple definitions".

`g()` — `inline`:

- *Can* be defined in a header and included in many sources.

`constexpr` **functions are implicitly** `inline` — safe in headers for free.

constexpr vs. inline

```
// constants.h
constexpr int square(int x) { return x * x; } // implicitly inline

constexpr double PI = 3.14159;   // internal linkage (own copy per TU)

inline constexpr double E = 2.71828; // external linkage, ONE copy (C++17)
```

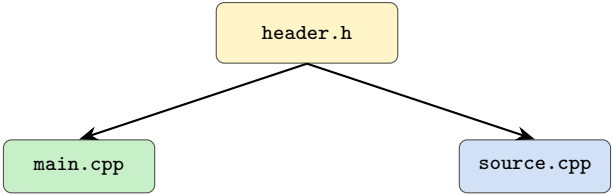
constexpr vs. inline

```
// constants.h
constexpr int square(int x) { return x * x; } // implicitly inline

constexpr double PI = 3.14159;   // internal linkage (own copy per TU)

inline constexpr double E = 2.71828; // external linkage, ONE copy (C++17)
```

Entity	Implicitly `inline`?	Safe in header?
`constexpr` function	yes	✓
`constexpr` variable	**no**	✓(own copy per TU)
`inline constexpr` variable	yes (C++17)	✓(shared copy)

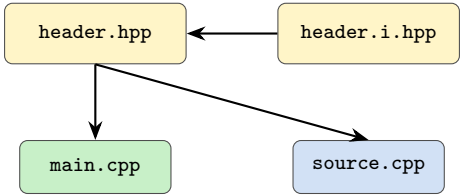
Code Structure #1 — One Header, Two Sources

- One header, two source files → **two translation units**.
- The header is included in *both* TUs.



Code Structure #2 — Interface + Implementation Header

- Two headers, two source files → **two translation units**.
- `.hpp` holds *declarations*; `.i.hpp` holds *implementations*.
- Both headers included in both TUs.



* Splitting declaration and implementation is optional — it just organises the code.

Standard Layout for a Class Across Files

student.h

```cpp
#pragma once
#include <string>

class Student {
public:
    Student(std::string n, int id);
    void print() const;
    static int count();
private:
    std::string name;
    int         id;
    static int  total;   // declaration
};
```

student.cpp

```cpp
#include "student.h"
#include <iostream>

// static member: defined ONCE here
int Student::total = 0;

Student::Student(std::string n, int i)
    : name(n), id(i) { ++total; }

void Student::print() const {
    std::cout << id << ": "
              << name << "\n";
}

int Student::count() { return total; }
```

Static Data Members Need Out-of-Class Definitions

A `static` data member declared *inside* the class body is only a **declaration**. Memory is allocated when you *define* it outside, in exactly one `.cpp`:

```
// student.h  (inside class body)
static int total;           // declaration -- no memory yet

// student.cpp
int Student::total = 0;     // definition -- memory allocated here
```

Static Data Members Need Out-of-Class Definitions

A `static` data member declared *inside* the class body is only a **declaration**. Memory is allocated when you *define* it outside, in exactly one `.cpp`:

```
// student.h  (inside class body)
static int total;           // declaration -- no memory yet

// student.cpp
int Student::total = 0;     // definition -- memory allocated here
```

C++17 exception: `inline static` members can be defined and initialised inside the class:

```
// student.h  (inside class body)
inline static int total = 0;  // definition + init, safe in header
```

#pragma once — Preventing Double Inclusion

A header included through multiple paths is pasted **twice** into one TU.
ODR Point 1 then fires ⇒ the class is defined twice.

Two solutions:

Modern

```
#pragma once    // one line, always works

class Student { /* ... */ };
```

Classic include guard

```
#ifndef STUDENT_H
#define STUDENT_H

class Student { /* ... */ };

#endif
```

`#pragma once` — Preventing Double Inclusion

A header included through multiple paths is pasted **twice** into one TU.
ODR Point 1 then fires ⇒ the class is defined twice.

Two solutions:

Modern

```
#pragma once    // one line, always works

class Student { /* ... */ };
```

Classic include guard

```
#ifndef STUDENT_H
#define STUDENT_H

class Student { /* ... */ };

#endif
```

Prefer `#pragma once`.
The classic guard breaks silently if you reuse the same macro name in two headers.

Predict the Output — ODR Undefined Behaviour

Both files define `inline int f()`, but with *different* bodies.

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

int g();   // defined in other.cpp

int main() {
    std::cout << f() << "\n"; // ?
    std::cout << g() << "\n"; // ?
}
```

other.cpp

```cpp
// Different body -- ODR VIOLATION!
inline int f() { return 5; }

int g() { return f(); }
```

Predict the Output — ODR Undefined Behaviour

Both files define `inline int f()`, but with *different* bodies.

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

int g();   // defined in other.cpp

int main() {
    std::cout << f() << "\n"; // ?
    std::cout << g() << "\n"; // ?
}
```

other.cpp

```cpp
// Different body -- ODR VIOLATION!
inline int f() { return 5; }

int g() { return f(); }
```

Output: `3 3` *or* `3 5` — the linker picks *one* definition arbitrarily. At `-O3` the compiler may inline `f()` into `g()`, changing the answer.

Predict the Output — ODR Undefined Behaviour

Both files define `inline int f()`, but with *different* bodies.

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

int g();   // defined in other.cpp

int main() {
    std::cout << f() << "\n"; // ?
    std::cout << g() << "\n"; // ?
}
```

other.cpp

```cpp
// Different body -- ODR VIOLATION!
inline int f() { return 5; }

int g() { return f(); }
```

Output: `3 3` *or* `3 5` — the linker picks *one* definition arbitrarily. At `-O3` the compiler may inline `f()` into `g()`, changing the answer.

Undefined behaviour: no warning, no error, just mysterious results.

Detecting ODR Violations

ODR violations are often *impossible* for the compiler to diagnose — but a few tools help:

- `-detect-odr-violations` — gold / LLVM linker
- `-Wodr -flto` — GCC
- Clang address sanitizer + `ASAN_OPTIONS=detect_odr_violation=2`

Detecting ODR Violations

ODR violations are often *impossible* for the compiler to diagnose — but a few tools help:

- `-detect-odr-violations` — gold / LLVM linker
- `-Wodr -flto` — GCC
- Clang address sanitizer + `ASAN_OPTIONS=detect_odr_violation=2`

Linkage Quick-Reference

Entity	**Specifier**	**Linkage**
Local variable	(none)	None
Global variable	(none)	External
Global variable	`static`	Internal
`const` global	(none)	Internal
`inline const` global	(none, C++17)	External
Function	(none)	External
Function	`static`	Internal
`inline` function	(none)	External (ODR Pt.3)
`constexpr` function	(none)	External (implicitly `inline`)

ODR Summary

Point	What it means
1	One definition per entity per **translation unit**. Caught by the **compiler**.
2	One definition per non-`inline` entity across the **whole program**. Caught by the **linker**. Put definitions in `.cpp`, declarations in `.h`.
3	`inline`, `template`, and types may appear in multiple TUs, but every copy must be **identical**. Violations are **undefined behaviour**.

What Goes in Headers vs. Source Files

Entity	**Header?**	**Reason**
Class / struct definition	✓	ODR Pt.3 (types)
Function *declaration*	✓	declaration only
`inline` function body	✓	ODR Pt.3
`template` definition	✓	ODR Pt.3
`const`/`constexpr` variable	✓	internal linkage
`inline constexpr` variable	✓(C++17)	one shared copy
Regular function *body*	×	linker error
Regular variable definition	×	linker error

What Goes in Headers vs. Source Files

Entity	**Header?**	**Reason**
Class / struct definition	✓	ODR Pt.3 (types)
Function *declaration*	✓	declaration only
`inline` function body	✓	ODR Pt.3
`template` definition	✓	ODR Pt.3
`const`/`constexpr` variable	✓	internal linkage
`inline constexpr` variable	✓(C++17)	one shared copy
Regular function *body*	×	linker error
Regular variable definition	×	linker error

Golden rule

If putting something in a header and including it in two `.cpp` files would hand the linker *two copies of the same external symbol*, it belongs in `.cpp`, not the header.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).
4. `static` on a *local* ⇒ persistent lifetime (same name, completely different effect).

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).
4. `static` on a *local* ⇒ persistent lifetime (same name, completely different effect).
5. `extern` *declares* a symbol defined elsewhere — no memory allocated.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).
4. `static` on a *local* ⇒ persistent lifetime (same name, completely different effect).
5. `extern` *declares* a symbol defined elsewhere — no memory allocated.
6. `const`/`constexpr` globals have *internal* linkage — each TU gets its own copy.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).
4. `static` on a *local* ⇒ persistent lifetime (same name, completely different effect).
5. `extern` *declares* a symbol defined elsewhere — no memory allocated.
6. `const`/`constexpr` globals have *internal* linkage — each TU gets its own copy.
7. **ODR:** define non-`inline` things exactly once. Headers hold declarations and `inline`/`template` bodies.

Summary

1. A **translation unit** = one `.cpp` + everything it includes. The compiler processes each TU in *isolation*.
2. **Linkage** controls symbol visibility: none / internal / external.
3. `static` on a *global* ⇒ internal linkage (file-private).
4. `static` on a *local* ⇒ persistent lifetime (same name, completely different effect).
5. `extern` *declares* a symbol defined elsewhere — no memory allocated.
6. `const`/`constexpr` globals have *internal* linkage — each TU gets its own copy.
7. **ODR:** define non-`inline` things exactly once. Headers hold declarations and `inline`/`template` bodies.
8. `inline` lets identical definitions live in many TUs; the linker keeps one.

Questions?